

Q1.

Problem Statement

In a busy airport, passengers are holding tickets with unique numbers.

Write a program using the linear search algorithm to check if a given ticket number exists in the array of ticket numbers.

If the ticket number is found, print a congratulatory message along with the position where it's found. If the ticket number is not found, encourage the passenger with a message for better luck on the next attempt.

Input Format

The first line of input consists of an integer **N**, representing the number of tickets. The second line consists of **N** space-separated integers, representing the ticket numbers.

The third line consists of an integer **X**, representing the ticket number to search.

Output Format

If the ticket number is found, print "Congratulations! Ticket number X found at position Y!", where X is the ticket number and Y is the position (position starts from 1).

Otherwise, print "Better luck next time! Ticket number X not found!", where X is the ticket number.

Refer to the sample output for formatting specifications.

Constraints

$1 \leq N \leq 10$

$10 \leq \text{ticket number, } X \leq 1000$

Sample Input Sample Output

5

123 456 789 321 654

321

Congratulations! Ticket number 321 found at position 4!

Sample Input Sample Output

6

741 456 982 364 789 258

369

Better luck next time! Ticket number 369 not found!

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main() {
5
6      int search, i, n;
7      scanf("%d", &n);
8
9      int *a = malloc(n * sizeof *a);
10     if (a == NULL) {
11         fprintf(stderr, "Memory allocation failed\n");
12         return 1;
13     }
14     for (i = 0; i < n; i++) {
15         scanf("%d", &a[i]);
16     }
17
18     scanf("%d", &search);
19
20     for (i = 0; i < n; i++) {
21         if (a[i] == search) {
22             printf("Congratulations! Ticket number %d found at position %d!\n", search, i + 1);
23             break;
24         }
25     }
26
27     if (i >= n) {
28         printf("Better luck next time! Ticket number %d not found!\n", search);
29     }
30     free(a);
31     return 0;
32 }
33

```

Q2.

Problem Statement

You are working as a data analyst for a company that collects user feedback scores for their products. The feedback scores are recorded sequentially, and sometimes you need to identify specific feedback points for analysis.

Given a list of feedback scores and a particular feedback score, your task is to find the position of the second occurrence of this feedback score in the list. If the feedback score appears less than twice, you need to handle that as well:

- If the feedback score does not appear at all, print a message indicating that the score was not found.

- If the feedback score appears only once, print a message indicating that the score was found only once.
- If the feedback score appears more than once, print the position (index) of the second occurrence.

Input Format

The first input line contains an integer **n**, the number of feedback scores.

The second input line contains **n** space-separated integers, each representing a feedback score.

The third input line contains an integer **X**, which is the feedback score you need to find the second occurrence of.

Output Format

The output displays the following format:

- The position of the second occurrence of the key in the list, or:
- "X Score not found." if the score is not present in the list, or:
- "X Score found only once." if the score appears only once.

Refer to the sample output for formatting specifications.

Constraints

$$1 \leq N \leq 10$$

$$0 \leq \text{score}, K \leq 100$$

Sample Input Sample Output

5

45 56 98 56 32

56

3

Sample Input Sample Output

4

71 52 63 94

89

89 Score not found.

Sample Input Sample Output

4

85 96 32 45

45

45 Score found only once.

```

1  #include<stdio.h>
2  #include<stdlib.h>
3
4  int main() {
5      int count = 0;
6      int search, i ,n;
7      printf("Enter the number of feedback entries: ");
8      scanf("%d", &n);
9      int *a = malloc(n *sizeof *a);
10     if (a ==NULL){
11         fprintf(stderr, "Memory allocation failed\n");
12         return 1;
13     }
14
15     printf("Enter the feedback points: ");
16     for (i = 0; i < n; i++) {
17         scanf("%d", &a[i]);
18     }
19
20     printf("Enter the feedback point to evaluate: ");
21     scanf("%d", &search);
22
23     for (i = 0; i < n; i++) {
24         if (a[i] == search) {
25             count++;
26             if (count == 2) {
27                 printf("The second occurrence of feedback point %d is at position %d.\n", search, i + 1);
28                 break;
29             }
30         }
31     }
32     if (count == 1) {
33         printf("The feedback point %d found only Once.\n", search);
34     } else if (count == 0) {
35         printf("The feedback point %d not found.\n", search);
36     }
37
38 }

```

Q3.

Problem Statement

In a local election, each candidate is assigned a unique numeric ID. After collecting the votes, you need to determine if any candidate received more than $n-2$ votes, where n is the total number of votes cast. This will help in identifying if there is a clear winner with significant support.

Your task is to write a program that determines if there is a candidate ID that received more than $n-2$ votes. If such a candidate exists, return their ID. Otherwise, return "No majority candidate found."

Input Format

The first line contains an integer n — the total number of votes cast.

The second line contains n space-separated integers, each representing the candidate ID for each vote.

Output Format

The output displays the following format:

If a candidate received more than $n-2$ votes, output their ID.

If no candidate meets this criterion, output "No majority candidate found."

Refer to the sample output for formatting specifications.

Constraints

$$1 \leq n \leq 10$$

$$1 \leq \text{candidate ID} \leq 100$$

Sample Input Sample Output

5

1 4 1 1 1

1

Sample Input Sample Output

8

12 12 12 12 13 12 12 12

12

Sample Input Sample Output

5

56 58 45 54 55

No majority candidate found.

```

1  #include<stdio.h>
2  #include<stdlib.h>
3
4  int main() {
5
6      int n, i, j, count;
7
8      scanf("%d", &n);
9
10     int *a = malloc(n * sizeof(int));
11
12     for(i = 0; i < n; i++)
13         scanf("%d", &a[i]);
14
15     for(i = 0; i < n; i++) {
16
17         count = 0;
18
19         for(j = 0; j < n; j++) {
20
21             if(a[i] == a[j])
22                 count++;
23         }
24
25         if(count > n - 2) {
26             printf("%d", a[i]);
27             free(a);
28             return 0;
29         }
30     }
31
32     printf("No majority candidate found.");
33
34     free(a);
35
36     return 0;
37 }

```

Q4.

Problem Statement

You are part of a legal team responsible for reviewing contracts and legal documents. To streamline the review process, you need a tool that can quickly check if specific legal terms or clauses are included in a large text document. This tool helps ensure that essential terms are not overlooked.

Write a program that searches a given document (a sentence) for the presence of a specific legal term or clause. The tool should indicate whether the term is present in the document.

Input Format

The first line consists of a string **S**, the document (a single sentence) containing up to 1000 characters.

The second line consists of a string **T**, representing the legal term or clause to search for, which can be up to 100 characters long.

Output Format

The output displays the following format:

If the legal term or clause is found in the document, output: The word 'T' is present in the given sentence.

If the legal term or clause is not found, output: The word 'T' is not present in the given sentence.

Refer to the sample output for formatting specifications.

Constraints

$1 \leq \text{length of } S \leq 999$

$1 \leq \text{length of } T \leq 99$

The input sentence is case-sensitive.

Sample Input Sample Output

Constellations of stars, murals on city walls
stars

The word 'stars' is present in the given sentence.

Sample Input Sample Output

Constellations of Stars, murals on city walls
stars

The word 'stars' is not present in the given sentence.

Q5.

Problem Statement

In a warehouse, inventory management requires checking the availability of specific items in stock. Each item in the inventory is identified by a unique numerical ID. Given a list of items currently in the warehouse and a set of item IDs that need to be checked, determine if each item ID from the set is present in the warehouse inventory and identify its position if it is available.

Implement a tool that reads the inventory IDs, followed by a list of item IDs to be checked, and then performs a search to determine if each item ID is present in the inventory. For each item ID, output its position in the inventory if found, or indicate that the item is not available.

Input Format

The first line contains an integer **n1**, representing the number of items in the warehouse inventory.

The second line contains n1 space-separated integers, representing the IDs of the items in the warehouse inventory.

The third line contains an integer **n2**, representing the number of item IDs to be checked.

The fourth line contains n2 space-separated integers, representing the item IDs that need to be checked.

Output Format

The output displays for each item ID in the checklist, separated by a line:

If the item ID is found in the inventory, output the item ID followed by its position (1-based index) in the inventory.

If the item ID is not found in the inventory, output the item ID followed by "not found in the inventory".

Refer to the sample outputs for format specifications.

Constraints

$$1 \leq n1 \ n2 \leq 10$$

The index starts at 1.

Sample Input Sample Output

5

10 20 30 40 50

4

20 30 40 60

20 found at position 2

30 found at position 3

40 found at position 4

60 not found in the inventory

Sample Input Sample Output

4

11 22 33 44

2

22 55

22 found at position 2

55 not found in the inventory



```

1  #include <stdio.h>
2  #include<stdlib.h>
3
4  int main(){
5      int n,m;
6      printf("Enter the size of Inventory: ");
7      scanf("%d",&n);
8
9      int *inventory = malloc(n * sizeof(int));
10     printf("Enter the items in the inventory: ");
11     for(int i =0; i<n; i++){
12         scanf("%d",&inventory[i]);
13     }
14
15     printf("enter the size of items to searvch: ");
16     scanf("%d",&m);
17     int *items = malloc(m * sizeof(int));
18     printf("Enter the items to search: ");
19     for(int i =0; i<m; i++){
20         scanf("%d",&items[i]);
21     }
22     //for each item in the items array, we will search it in the inventory array
23     for(int i = 0 ; i < m; i++){
24         int j; // declare j outside the loop to use it after the loop
25         for( j = 0 ; j < n; j++){
26             if(items[i] == inventory[j]){
27                 printf("Item %d found at index %d\n",items[i],j+1);
28                 break;
29             }
30         }
31         // naturally, if the loop completes without finding the item, j will be equal to n
32         if(j == n){
33             printf("Item %d not found in the inventory\n",items[i]);
34         }
35     }
36
37     free(inventory);
38     free(items);
39     return 0;
40 }

```

Q6.

Problem Statement

In a company's customer service center, each customer interaction is logged with a unique numerical ID. A customer support representative needs to search through these logs to find the position of a specific customer interaction ID and also analyze the overall interaction data.

Implement a tool to search for a specific customer interaction ID in a log of interaction IDs, determine its position if it exists, and report the total number of comparisons made during the search. Additionally, calculate and display the total number of interactions and the average number of interactions per ID.

Input Format

The first line contains an integer **n**, representing the number of customer interactions recorded.

The second line contains **n** integers, each representing the unique interaction ID recorded.

The third line contains an integer **x**, representing the interaction ID to be searched.

Output Format

The output displays the following format:

1. Print an integer the position (0-based index) of the interaction ID **x** if found. If not found, print a message stating that the interaction ID **x** is not present.
2. Print an integer the total number of comparisons made during the search process, prefixed by "Total comparisons made: ".
3. Print an integer the total sum of all interaction IDs, prefixed by "Sum of all interaction: ".
4. Print double value the average of all interaction IDs, prefixed by "Average of all interaction: ".

Refer to the sample output for formatting specifications.

Constraints

$$1 \leq n \leq 10$$

$$1 \leq \text{interaction ID} \leq 10^4$$

Sample Input Sample Output

5

40 20 50 30 10

30

Interaction ID 30 is present at index 3

Total comparisons made: 4

Sum of all interaction: 150

Average of all interaction: 30.00

Sample Input Sample Output

5

40 20 50 30 10

80

Interaction ID 80 is not present

Total comparisons made: 5

Sum of all interaction: 150

Average of all interaction: 30.00

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main() {
5      int n, x;
6      printf("Enter the number of interactions: ");
7      scanf("%d", &n);
8
9      int *arr = malloc(n * sizeof(int));
10
11     int sum = 0;
12     printf("Enter the interaction IDs: ");
13     // Read array and calculate sum
14     for (int i = 0; i < n; i++) {
15         scanf("%d", &arr[i]);
16         sum += arr[i];
17     }
18     printf("Enter the interaction ID to search: ");
19     scanf("%d", &x);
20
21     int comparisons = 0;
22     int index = -1;
23
24     // Linear Search
25     for (int i = 0; i < n; i++) {
26         comparisons++;
27
28         if (arr[i] == x) {
29             index = i;
30             break;
31         }
32     }
33
34     if (index != -1)
35         printf("Interaction ID %d is present at index %d\n", x, index);
36     else
37         printf("Interaction ID %d is not present\n", x);
38
39     printf("Total comparisons made: %d\n", comparisons);
40     printf("Sum of all interaction: %d\n", sum);
41     printf("Average of all interaction: %.2f\n", (double)sum / n);
42
43     free(arr);
44
45     return 0;
46 }
47 /*
48 Enter the number of interactions: 5
49 Enter the interaction IDs: 10
50 30
51 40
52 20
53 10
54 Enter the interaction ID to search: 10
55 Interaction ID 10 is present at index 0
56 Total comparisons made: 1
57 Sum of all interaction: 110
58 Average of all interaction: 22.00
59 */

```

Q7.

Problem Statement

Sathish is developing a simple inventory management system for a store. The store owner wants to update the stock levels of certain products based on the latest shipment.

Write a program that takes the current stock levels as input, along with the product code to search for and the new quantity received. Implement a linear search program to find the product in the inventory and update its stock with the new quantity.

Finally, print the updated inventory after the stock update.

Input Format

The first line consists of an integer **N**, representing the size of the inventory.

The second line consists of **N** space-separated integers, representing the current stock levels.

The third line consists of an integer **X**, representing the product code to search for.

The fourth line consists of an integer **M**, representing the new quantity received.

Output Format

The output prints the updated array with the **X** replaced by **M**. Elements should be separated by a space.

If the **X** is not found in the array, the output will be the same as the original array since no replacements are made.

Refer to the sample output for formatting specifications.

Constraints

$$1 \leq N \leq 10$$

$$10 \leq \text{stock level} \leq 100$$

$$10 \leq M \leq 100$$

Sample Input Sample Output

5

45 56 78 89 65

78

85

45 56 85 89 65

Sample Input Sample Output

8

73 91 45 67 98 36 22 49

73

75

75 91 45 67 98 36 22 49

Sample Input Sample Output

4

76 54 23 46

87

78

76 54 23 46

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main() {
5      int n;
6
7      printf("Enter the size of the inventory: ");
8      scanf("%d", &n);
9
10     int *arr = (int *)malloc(n * sizeof(int));
11
12     printf("Enter the inventory elements: ");
13     for (int i = 0; i < n; i++) {
14         scanf("%d", &arr[i]);
15     }
16
17     int x, m;
18
19     printf("Enter the product code to search: ");
20     scanf("%d", &x);
21
22     printf("Enter the new quantity/value: ");
23     scanf("%d", &m);
24
25     // Linear Search and Update
26     for (int i = 0; i < n; i++) {
27         if (arr[i] == x) {
28             arr[i] = m;
29             break;      // Remove break to replace all occurrences
30         }
31     }
32
33     printf("Updated Inventory:\n");
34     for (int i = 0; i < n; i++) {
35         printf("%d ", arr[i]);
36     }
37
38     printf("\n");
39
40     free(arr);
41     return 0;
42 }/*
43 Enter the size of the inventory: 4
44 Enter the inventory elements: 10
45 40
46 30
47 40
48 Enter the product code to search: 40
49 Enter the new quantity/value: 20
50 Updated Inventory:
51 10 20 30 40 */

```

Q8.

Problem Statement

In a manufacturing plant, quality control inspectors need to verify that all parts on a production line meet specific quality standards. Each part is assigned an ID number, and for the parts to be approved, all IDs must be even numbers.

Your task is to develop a tool that helps in checking whether all parts in a batch meet this requirement.

Given a list of part IDs, determine if every ID in the list is an even number. If all IDs are even, report that the batch is approved; otherwise, report that the batch is not approved.

Company Tags: DXC and TCS.

Input Format

The first line of input consists of an integer N , representing the list size.

The second line consists of N space-separated integers, representing the list of part IDs.

Output Format

The output displays the following format:

If all IDs are even, print: "All parts in the batch are approved."

Otherwise, print: "The batch is not approved."

Refer to the sample output for formatting specifications.

Constraints

$$1 \leq N \leq 10$$

$$1 \leq \text{part ID} \leq 100$$

Sample Input Sample Output

5

12 18 24 16 38

All parts in the batch are approved.

Sample Input Sample Output

3

15 26 39

The batch is not approved.



```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main() {
5      int n;
6
7      printf("Enter the number of part IDs: ");
8      scanf("%d", &n);
9
10     int *arr = (int *)malloc(n * sizeof(int));
11
12     printf("Enter the part IDs: ");
13     for (int i = 0; i < n; i++) {
14         scanf("%d", &arr[i]);
15     }
16
17     int approved = 1;
18
19     for (int i = 0; i < n; i++) {
20         if (arr[i] % 2 != 0) {
21             approved = 0;
22             break;
23         }
24     }
25
26     if (approved)
27         printf("All parts in the batch are approved.\n");
28     else
29         printf("The batch is not approved.\n");
30
31     free(arr);
32     return 0;
33 }/*
34 Enter the number of part IDs: 3
35 Enter the part IDs: 12
36 14
37 15
38 The batch is not approved.*/
```

Q9.

Problem Statement

Alex, a logistics manager, is responsible for handling various pieces of luggage. He needs to determine the weight distribution among these pieces and find the heaviest and lightest pieces, along with the weight difference between them.

To accomplish this, he plans to create a simple program that takes as input the weights of various pieces of luggage and uses the **linear search** algorithm to search and identify the heaviest and lightest pieces, as well as calculate the weight difference between them.

Write a program to help Alex accomplish this task.

Input Format

The first line consists of an integer **N**, representing the number of luggage.

The second line consists of **N** space-separated integers, representing the weights of the luggage.

Output Format

The first line of output prints "Heaviest piece weight: " followed by the weight of the heaviest piece.

The second line prints "Lightest piece weight: " followed by the weight of the lightest piece.

The third line prints "Weight difference: " followed by their weight difference.

Refer to the sample output for formatting specifications.

Constraints

The test cases fall under the following constraints:

$$1 \leq N \leq 10$$

$$1 \leq \text{weight of each piece} \leq 100$$

Sample Input Sample Output

5

10 15 20 25 30

Heaviest piece weight: 30

Lightest piece weight: 10

Weight difference: 20

Sample Input Sample Output

8

70 65 80 75 60 55 50 45

Heaviest piece weight: 80

Lightest piece weight: 45

Weight difference: 35



```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main() {
5      int n;
6
7      printf("Enter the number of luggage: ");
8      scanf("%d", &n);
9
10     int *arr = (int *)malloc(n * sizeof(int));
11
12     printf("Enter the luggage weights: ");
13     for (int i = 0; i < n; i++) {
14         scanf("%d", &arr[i]);
15     }
16
17     int max = arr[0];
18     int min = arr[0];
19
20     for (int i = 1; i < n; i++) {
21         if (arr[i] > max)
22             max = arr[i];
23
24         if (arr[i] < min)
25             min = arr[i];
26     }
27
28     printf("Heaviest piece weight: %d\n", max);
29     printf("Lightest piece weight: %d\n", min);
30     printf("Weight difference: %d\n", max - min);
31
32     free(arr);
33     return 0;
34 }/*
35 Enter the number of luggage: 4
36 Enter the luggage weights: 4
37 2
38 3
39 5
40 Heaviest piece weight: 5
41 Lightest piece weight: 2
42 Weight difference: 3*/
```

Q10.

Problem Statement

In a research lab specializing in data collection, the team frequently deals with large datasets where quick lookup and data reversal are crucial.

The team needs to ensure that certain data points (identified by unique IDs) are present in their datasets and also needs to analyze data trends by reversing the order of the collected data.

Input Format

The first line consists of an integer N representing the number of data points in the dataset.

The second line consists of the list of N integers representing the data points.

The third line consists of an integer x that represents the data point to be searched for in the dataset.

Output Format

The first line displays "Data point isn't present in the dataset" if the data point x is not found in the dataset. Otherwise, print "Data point is present in the dataset at position [index]" where index is the 0-based index of the data point in the dataset.

The second line displays the reversed dataset, with each data point separated by a space.

Refer to the sample output for formatting specifications.

Constraints

$$1 \leq N \leq 10$$

$$0 \leq \text{elements}, x \leq 100$$

Sample Input Sample Output

5

45 56 98 97 32

56

Data point is present in the dataset at position 1

32 97 98 56 45

Sample Input Sample Output

4

71 52 63 94

89

Data point isn't present in the dataset

94 63 52 71

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main() {
5      int n;
6
7      printf("Enter the number of data points: ");
8      scanf("%d", &n);
9
10     int *arr = (int *)malloc(n * sizeof(int));
11
12     printf("Enter the data points: ");
13     for (int i = 0; i < n; i++) {
14         scanf("%d", &arr[i]);
15     }
16
17     int x;
18     printf("Enter the data point to search: ");
19     scanf("%d", &x);
20
21     // Linear Search
22     int index = -1;
23
24     for (int i = 0; i < n; i++) {
25         if (arr[i] == x) {
26             index = i;
27             break;
28         }
29     }
30
31     if (index == -1)
32         printf("Data point isn't present in the dataset\n");
33     else
34         printf("Data point is present in the dataset at position %d\n", index);
35
36     // Reverse the array (in-place)
37     int left = 0, right = n - 1;
38
39     while (left < right) {
40         int temp = arr[left];
41         arr[left] = arr[right];
42         arr[right] = temp;
43
44         left++;
45         right--;
46     }
47
48     // Print reversed array
49     for (int i = 0; i < n; i++) {
50         printf("%d ", arr[i]);
51     }
52     printf("\n");
53
54     free(arr);
55     return 0;
56 }/*
57 Enter the number of data points: 4
58 Enter the data points: 45
59 56
60 25
61 36
62 Enter the data point to search: 25
63 Data point is present in the dataset at position 2
64 36 25 56 45 */

```

Q11.

Problem Statement

In a large-scale sales database, the management team is tasked with performing critical data analysis to ensure optimal business decisions. The database contains records of sales amounts for various transactions. For strategic planning, the team needs to:

- Quickly check if a particular sales amount has been recorded and, if so, identify its position in the dataset.
- To understand competitive performance and identify the second-best sales figures, which may be crucial for benchmarking and incentive planning.

Input Format

The first line consists of an integer **N** representing the number of transactions.

The second line consists of the list of **N** integers representing the sales amounts.

The third line consists of an integer **x** representing the sales amount to search for in the dataset.

Output Format

The output is displayed in the following format:

If the sales amount **x** is found in the array, print a message in the format:

"Sales amount x is recorded at position <pos>", where <pos> is the 1-based index of the sales amount in the dataset.

If the sales amount **x** is not found, print a message in the format:

"Sales amount x is not recorded in the dataset".

The second-highest sales amount in the dataset. If no second-highest amount exists, print:

"No second highest sales amount found".

Refer to the sample output for the formatting specifications.

Constraints

$$1 \leq n \leq 100$$

$$1 \leq \text{sales amount} \leq 100$$

$$1 \leq x \leq 100$$

Sample Input Sample Output

5

19 15 24 43 37

24

Sales amount 24 is recorded at position 3

37

Sample Input Sample Output

5

19 15 24 43 37

16

Sales amount 16 is not recorded in the dataset

37

Sample Input Sample Output

1

2

3

Sales amount 3 is not recorded in the dataset

No second highest sales amount found


```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <limits.h>
4
5  int main() {
6      int n;
7
8      printf("Enter the number of transactions: ");
9      scanf("%d", &n);
10
11     int *arr = (int *)malloc(n * sizeof(int));
12
13     printf("Enter the sales amounts: ");
14     for (int i = 0; i < n; i++) {
15         scanf("%d", &arr[i]);
16     }
17
18     int x;
19     printf("Enter the sales amount to search: ");
20     scanf("%d", &x);
21
22     // Linear Search
23     int index = -1;
24
25     for (int i = 0; i < n; i++) {
26         if (arr[i] == x) {
27             index = i;
28             break;
29         }
30     }
31
32     if (index == -1)
33         printf("Sales amount %d is not present\n", x);
34     else
35         printf("Sales amount %d is present at index %d\n", x, index);
36
37     // Find largest and second largest
38     int largest = INT_MIN;
39     int secondLargest = INT_MIN;
40
41     for (int i = 0; i < n; i++) {
42
43         if (arr[i] > largest) {
44             secondLargest = largest;
45             largest = arr[i];
46         }
47         else if (arr[i] > secondLargest && arr[i] != largest) {
48             secondLargest = arr[i];
49         }
50     }
51
52     if (secondLargest == INT_MIN)
53         printf("Second largest element does not exist\n");
54     else
55         printf("Second largest sales amount: %d\n", secondLargest);
56
57     free(arr);
58     return 0;
59 }/*
60 Enter the number of transactions: 5
61 Enter the sales amounts: 45
62 12
63 78
64 6
65 40
66 Enter the sales amount to search: 6
67 Sales amount 6 is present at index 3
68 Second largest sales amount: 45*/

```

Q12.

Arsh is working on enhancing a recommendation system for a shopping application. He needs to develop a feature that suggests the closest product prices to a target price entered by a user. The list of product prices is sorted in non-decreasing order, and he wants the system to find the K closest prices to the target price. If two prices have the same difference from the target, the lower price should be preferred.

To implement this feature, Arsh needs to write a program that identifies and prints the K closest prices to the given target price from the sorted list of prices. If there are two prices with the same difference value, print the larger price.

Example

Input:

```
5
2 4 7 8 10
5
3
```

Output:

```
4 7 8
```

Explanation

The closest crossover point for $x = 5$ is at index 1 (element 4).

Compare differences: 4 ($|5-4|=1$), 7 ($|7-5|=2$), 8 ($|8-5|=3$).

Closest $k=3$ elements are 4, 7, 8 based on proximity to $x=5$.

Input Format

The first line contains an integer **n**, representing the number of product prices.

The second line contains **n** space-separated integers, representing the sorted product prices.

The third line contains an integer **x**, denoting the target price.

The fourth line contains an integer **k**, representing the number of closest prices to find.

Output Format

The output displays k space-separated integers representing the k closest prices to the target value.

Refer to the sample output for the formatting specifications.

Constraints

$1 \leq n \leq 10$

$1 \leq \text{product prices} \leq 100$

Prices are given in ascending order.

$1 \leq k \leq n$

Sample Input Sample Output

5

2 4 7 8 10

5

3

4 7 8

Sample Input Sample Output

8

1 3 5 7 9 11 13 15

10

1

11

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  // Function to find the crossover point
5  // Crossover point = largest element <= x
6  int findCrossover(int arr[], int low, int high, int x) {
7
8      // If every element is <= x
9      if (arr[high] <= x)
10         return high;
11
12     // If every element is > x
13     if (arr[low] > x)
14         return low;
15
16     // Binary Search
17     while (low <= high) {
18
19         int mid = (low + high) / 2;
20
21         // Found crossover point
22         if (arr[mid] <= x && arr[mid + 1] > x)
23             return mid;
24
25         // Move right
26         if (arr[mid] < x)
27             low = mid + 1;
28
29         // Move left
30         else
31             high = mid - 1;
32     }
33
34     return high;
35 }
36
37 int main() {
38
39     int n;
40
41     printf("Enter number of elements: ");
42     scanf("%d", &n);
43
44     int *arr = (int *)malloc(n * sizeof(int));
45
46     printf("Enter sorted array: ");
47     for (int i = 0; i < n; i++)
48         scanf("%d", &arr[i]);
49
50     int x, k;
51
52     printf("Enter target value: ");
53     scanf("%d", &x);
54
55     printf("Enter K: ");
56     scanf("%d", &k);
57
58     // Find the crossover point
59     int left = findCrossover(arr, 0, n - 1, x);
60
61     // Right pointer starts immediately after crossover
62     int right = left + 1;
63
64     // If x itself exists, ignore it
65     if (left >= 0 && arr[left] == x)
66         left--;
67
68     printf("Closest %d elements are:\n", k);
69
70     // Compare left and right elements
71     while (left >= 0 && right < n && k > 0) {
72
73         // Distance from target
74         int leftDiff = x - arr[left];
75         int rightDiff = arr[right] - x;
76
77         // Left element is closer
78         if (leftDiff < rightDiff) {
79             printf("%d ", arr[left]);
80             left--;
81         }
82
83         // Right element is closer
84         else if (leftDiff > rightDiff) {
85             printf("%d ", arr[right]);
86             right++;
87         }
88
89         // Equal difference
90         else {
91
92             // Printing larger value
93             // (Change to left if question wants smaller value)
94             printf("%d ", arr[right]);
95             right++;
96         }
97
98         k--;
99     }
100
101     // If left side still has elements
102     while (k > 0 && left >= 0) {
103         printf("%d ", arr[left]);
104         left--;
105     }
106
107     // If right side still has elements
108     while (k > 0 && right < n) {
109         printf("%d ", arr[right]);
110         right++;
111     }
112
113     printf("\n");
114
115     free(arr);
116
117     return 0;
118 }
119
120 /*
121 Enter number of elements: 5
122 Enter sorted array: 2
123 4
124 7
125 8
126 10
127 Enter target value: 8
128 Enter K: 2
129 Closest 2 elements are:
130 7 10 */

```

Q13.

You are in charge of preparing a series of meals for a special event. Each meal needs to be cooked using ingredients from a set of ingredient boxes. Each box becomes available on a specific day, and you need to cook a certain number of meals using the ingredients from adjacent boxes.

You are given an integer array `availabilityDay`, an integer `m`, and an integer `k`.

- `availabilityDay[i]` represents the day when the *i*-th ingredient box becomes available.
- To prepare one meal, you need to use `k` adjacent ingredient boxes.
- You need to prepare `m` meals.

Your task is to find the minimum number of days you need to wait to be able to prepare `m` meals using the available ingredient boxes. If it's impossible to prepare the required number of meals, return -1. Implement the problem using recursive binary search.

Example 1

Input:

```
5
1 10 3 10 2
3
1
```

Output:

```
3
```

Explanation:

You need to prepare 3 dishes, each needing 1 box of ingredients. The boxes become available on different days:

Day 1: [x, _, _, _, _] - 1 dish possible.

Day 2: [x, _, _, _, x] - 2 dishes possible.

Day 3: [x, _, x, _, x] - 3 dishes possible.

The minimum number of days required is 3.

Example 2

Input:

```
5
1 10 3 10 2
3
2
```

Output:

```
-1
```

Explanation:

You need to prepare 3 dishes, each requiring 2 boxes of ingredients. To make 3 dishes, you need a total of 6 boxes. Since there are only 5 boxes available, it is impossible to prepare all 3 dishes. Therefore, the output is -1.

Input Format

The first line contains an integer **n**, representing the number of ingredient boxes. The second line contains **n** space-separated integers, representing the days when each ingredient box becomes available.

The third line contains an integer **m**, representing the number of meals needed.

The fourth line contains an integer **k**, representing the number of adjacent ingredient boxes required for one meal.

Output Format

The output prints the minimum number of days required to be able to prepare **m** meals, or -1 if it is not possible.

Refer to the sample output for formatting specifications.

Constraints

$$1 \leq n \leq 100$$

$$1 \leq m \leq 100$$

$$1 \leq k \leq n$$

Sample Input Sample Output

```
5
1 10 3 10 2
3
1
3
```

Sample Input Sample Output

```
5
1 10 3 10 2
3
2
-1
```

Sample Input Sample Output

7

7 7 7 7 12 7 7

2

3

12

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 // Check whether m meals can be prepared on the given day
5 int canPrepareMeals(int availabilityDay[], int size, int day, int meals, int boxesNeeded) {
6
7     int consecutiveBoxes = 0;
8     int mealsPrepared = 0;
9
10    for (int i = 0; i < size; i++) {
11
12        // Box is available
13        if (availabilityDay[i] <= day) {
14            consecutiveBoxes++;
15
16            // Enough adjacent boxes for one meal
17            if (consecutiveBoxes == boxesNeeded) {
18                mealsPrepared++;
19                consecutiveBoxes = 0;
20            }
21        }
22        else {
23            consecutiveBoxes = 0;
24        }
25    }
26
27    return mealsPrepared >= meals;
28 }
29
30 // Recursive Binary Search
31 int recursiveBinarySearch(int availabilityDay[], int size,
32                             int low, int high,
33                             int meals, int boxesNeeded) {
34
35     if (low > high)
36         return low;
37
38     int mid = (low + high) / 2;
39
40     if (canPrepareMeals(availabilityDay, size, mid, meals, boxesNeeded))
41         return recursiveBinarySearch(availabilityDay, size,
42                                         low, mid - 1,
43                                         meals, boxesNeeded);
44     else
45         return recursiveBinarySearch(availabilityDay, size,
46                                         mid + 1, high,
47                                         meals, boxesNeeded);
48 }
49
50 int main() {
51
52     int size;
53
54     printf("Enter number of ingredient boxes: ");
55     scanf("%d", &size);
56
57     int *availabilityDay = (int *)malloc(size * sizeof(int));
58
59     printf("Enter availability days:\n");
60     for (int i = 0; i < size; i++)
61         scanf("%d", &availabilityDay[i]);
62
63     int meals, boxesNeeded;
64
65     printf("Enter number of meals: ");
66     scanf("%d", &meals);
67
68     printf("Enter boxes needed per meal: ");
69     scanf("%d", &boxesNeeded);
70
71     // Impossible case
72     if (meals * boxesNeeded > size) {
73         printf("-1\n");
74         free(availabilityDay);
75         return 0;
76     }
77
78     // Find maximum day
79     int maxDay = availabilityDay[0];
80
81     for (int i = 1; i < size; i++) {
82         if (availabilityDay[i] > maxDay)
83             maxDay = availabilityDay[i];
84     }
85
86     int answer = recursiveBinarySearch(
87         availabilityDay,
88         size,
89         1,
90         maxDay,
91         meals,
92         boxesNeeded
93     );
94
95     printf("Minimum number of days = %d\n", answer);
96
97     free(availabilityDay);
98
99     return 0;
100 }

```



```
PS A:\VIT Chennai\DSAA_01> & "A:\VIT Chennai\DSAA_01\DSA
Enter number of ingredient boxes: 5
Enter availability days:
1
10
3
10
2
Enter number of meals: 3
Enter boxes needed per meal: 1
Minimum number of days = 3
```

Q.14

You are part of a team managing an amusement park with several rides, each identified by a unique ride ID. These ride IDs are stored in a sorted list in ascending order. When a new ride is introduced or when the system needs to locate a ride, you need to find the correct position of the ride ID in the list.

If the ride ID already exists, return its position. If not, return the index where the new ride ID should be inserted to maintain the order.

Input Format

The first line contains an integer **n**, representing the number of existing ride IDs in the sorted list.

The second line contains **n** space-separated integers, representing the unique IDs of the rides in ascending order.

The third line contains an integer **targetID**, representing the ID of the new ride to be introduced or located.

Output Format

The output is a single integer representing the index where the targetID would be inserted in the sorted list of ride IDs to maintain the order.

Refer to the sample output for formatting specifications.

Constraints

$$1 \leq n \leq 100$$

$$1 \leq \text{unique IDs of the rides} \leq 100$$

$$1 \leq \text{targetID} \leq 100$$

Sample Input Sample Output

4

1 3 5 6

5

2

Sample Input Sample Output

4

1 3 5 6

2

1

Sample Input Sample Output

3

10 20 30

40

3

Sample Input Sample Output

3

10 20 40

1

0

Q15.

Problem Statement

You are working as a software developer at a financial analytics company. The company provides tools for various financial analyses, including computing statistics on sets of data.

Today, you are assigned to enhance a tool that calculates the median of two datasets. These datasets represent financial transactions recorded over different periods. Your task is to develop a feature that finds the median of the combined data, which helps in understanding the central tendency of transaction amounts over time.

Input Format

The first line contains the number of data points in the first dataset.

The second line contains the data points of the first dataset in sorted order.

The third line contains the number of data points in the second dataset.

The fourth line contains the data points of the second dataset in sorted order.

Output Format

The output prints the median of the combined datasets, as a float value rounded to two decimal places.

Refer to the sample output for formatting specifications.

Constraints

$1 \leq \text{number of data points in the first dataset} \leq 10$

$1 \leq \text{number of data points in the second dataset} \leq 10$

$1 \leq \text{data points} \leq 100$

Sample Input Sample Output

3

10 20 30

2

10 20

20.00

Sample Input Sample Output

5

1 2 3 4 5

2

6 7

4.00

Sample Input Sample Output

3

54 87 90

1

67

77.00

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main() {
5
6      int size1, size2;
7
8      // Read size of first array
9      printf("Enter the size of the first array: ");
10     scanf("%d", &size1);
11
12     int *firstArray = (int *)malloc(size1 * sizeof(int));
13
14     // Read first sorted array
15     printf("Enter the elements of the first sorted array:\n");
16     for (int i = 0; i < size1; i++)
17         scanf("%d", &firstArray[i]);
18
19     // Read size of second array
20     printf("Enter the size of the second array: ");
21     scanf("%d", &size2);
22
23     int *secondArray = (int *)malloc(size2 * sizeof(int));
24
25     // Read second sorted array
26     printf("Enter the elements of the second sorted array:\n");
27     for (int i = 0; i < size2; i++)
28         scanf("%d", &secondArray[i]);
29
30     // Allocate memory for merged array
31     int *mergedArray = (int *)malloc((size1 + size2) * sizeof(int));
32
33     // Indices for first array, second array, and merged array
34     int firstIndex = 0;
35     int secondIndex = 0;
36     int mergedIndex = 0;
37
38     // Merge both sorted arrays
39     while (firstIndex < size1 && secondIndex < size2) {
40
41         if (firstArray[firstIndex] < secondArray[secondIndex])
42             mergedArray[mergedIndex++] = firstArray[firstIndex++];
43         else
44             mergedArray[mergedIndex++] = secondArray[secondIndex++];
45     }
46
47     // Copy remaining elements from first array
48     while (firstIndex < size1)
49         mergedArray[mergedIndex++] = firstArray[firstIndex++];
50
51     // Copy remaining elements from second array
52     while (secondIndex < size2)
53         mergedArray[mergedIndex++] = secondArray[secondIndex++];
54
55     // Total number of elements
56     int totalElements = size1 + size2;
57
58     float median;
59
60     // Calculate median
61     if (totalElements % 2 == 0)
62         median = (mergedArray[totalElements / 2] +
63                 mergedArray[totalElements / 2 - 1]) / 2.0;
64     else
65         median = mergedArray[totalElements / 2];
66
67     // Display result
68     printf("Median of the merged arrays = %.2f\n", median);
69
70     // Free allocated memory
71     free(firstArray);
72     free(secondArray);
73     free(mergedArray);
74
75     return 0;
76 }
77 /*
78 PS A:\VIT Chennai\DSAA_01\DSA - Lab\Lab-01\
79 Program-15-Merge-Two-Sorted-Arrays-Median>
80 & "A:\VIT Chennai\DSAA_01\DSA - Lab\Lab-01\
81 Program-15-Merge-Two-Sorted-Arrays-Median\
82 main.exe"
83 Enter the size of the first array: 3
84 Enter the elements of the first sorted array:
85 54
86 87
87 90
88 Enter the size of the second array: 1
89 Enter the elements of the second sorted array:
90 67
91 Median of the merged arrays = 77.00*/

```

